

Experiment-2

Aim: Create IPC, Messaging, and Publish/Subscribe –

Description:

1)Inter-Process Communication (IPC)

Inter-Process Communication (IPC) encompasses a collection of techniques that allow independent processes to communicate and synchronize with one another. Because processes typically execute in separate memory spaces and operate autonomously, IPC mechanisms provide controlled and efficient methods for data exchange and coordination.

Common IPC Mechanisms

- **Pipes:**Enable one-way communication between processes, commonly used for data transfer between a parent process and its child.
- **Named Pipes (FIFOs):**Function similarly to standard pipes but allow communication between unrelated processes by using a named file system entry.
- **Message Queues:**Facilitate communication by allowing processes to send and receive structured messages in a queued manner.
- **Shared Memory:**The fastest IPC method, where multiple processes access a shared memory region directly for high-performance data exchange.
- **Semaphores and Mutexes:**Synchronization tools used to manage access to shared resources and prevent race conditions.
- **Sockets:**Support communication between processes running on the same system or across different networked systems.

Use Cases

- Operating system services and kernel-level process management
- Client-server communication in distributed systems
- Synchronization and coordination among multiple processes
- Real-time data exchange in performance-critical applications

Creating IPC

Step 1: Setup Your Environment

Check by running: Python –version

Create a working directory for the lab and change to your directory: Mkdir week2

Cd Week2

Step 2: Pipes IPC Example.

Create two Python files in your week2 folder:

1. write the following code in writer.py file:

```
import os, sys
```

```
message = "Hello from writer to the Reader!"
```

```
os.write(sys.stdout.fileno(), message.encode())//consumer.py
```

2. write the following code in reader.py file:

```
import sys
```

```
data = sys.stdin.buffer.read()
```

```
print("Reader received the data from Sender:", data.decode())//producer.py
```

3. Run them together in one terminal by piping output from consumer to producer:

```
python consumer.py | python producer.py
```

```
E:\24485A0503\cc_lab1\Week2>python consumer.py | python producer.py
Reader received the data from Sender: Hello from writer to the Reader!
E:\24485A0503\cc_lab1\Week2>
```

Shared Memory IPC

Shared Memory IPC is a method of inter-process communication in which two or more processes access a common memory region to exchange data efficiently.

- This approach allows processes to read from and write to the same memory segment, enabling extremely fast data transfer.

Create a Python script named `memory_shared.py` in your working directory:

Write the following code inside `memory_shared.py` file

```
from multiprocessing import Process, Value
```

```
import time
```

```
def writer(val):
```

```
    for i in range(8):
```

```
        val.value = i
```

```
        print("Writer wrote:", i)
```

```
        time.sleep(1)
```

```
def reader(val):
```

```
    last = -1
```

```
    while True:
```

```
        if val.value != last:
```

```
            print("Reader read:", val.value)
```

```
            last = val.value
```

```
        if val.value == 4:
```

```
            break
```

```
if __name__ == "__main__":
```

```
    number = Value('i', 0)
```

```
    p1 = Process(target=writer, args=(number,))
```

```
    p2 = Process(target=reader, args=(number,))
```

```
    p1.start()
```

```
    p2.start()
```

```
    p1.join()
```

```
    p2.join()
```

Run the script in your terminal (within your week2 folder)

python memory_shared.py

```
E:\24485A0503\cc_lab1\Week2>python memory_shared.py
Writer wrote: 0
Reader read: 0
Reader read: 1
Writer wrote: 1
Writer wrote: 2
Reader read: 2
Writer wrote: 3
Reader read: 3
Writer wrote: 4
Writer wrote: 5
Writer wrote: 6
Writer wrote: 7
E:\24485A0503\cc_lab1\Week2>
```

2)Messaging IPC

Messaging is a method of inter-process communication where processes or systems exchange information in the form of discrete messages rather than through shared memory.

Key Characteristics

- **Asynchronous** – The sender and receiver do not need to operate simultaneously.
- **Reliable Delivery** – Depending on the system, messages are stored until successfully received by the recipient.
- **Decoupled Communication** – Processes do not need direct knowledge of each other, promoting modularity.

Examples

- **Message Queues** – POSIX or System V implementations
- **Middleware Solutions** – RabbitMQ, ZeroMQ, Kafka
- **OS-level Mailboxes** – Mechanisms provided by the operating system for message passing

Advantages

- Loose coupling between communicating processes
- High scalability for growing systems
- Simplified error handling
- Well-suited for distributed systems and networked applications

Create Message Queue IPC :

Create a Python script named queue_message.py and write the following code inside it.

```
from multiprocessing import Process, Queue
```

```
import time
```

```
def producer(q):
```

```
    messages = ["one", "two", "three", "four", "five"]
```

```
    for msg in messages:
```

```
        print("Producer sent:",msg)
```

```
        q.put(msg)
```

```
        time.sleep(1)
```

```

q.put("END")
def consumer(q):
    while True:
        msg = q.get()
        print("Consumer received:", msg)
        if msg == "END":
            break
if __name__ == "__main__":
    q = Queue()
    p1 = Process(target=producer, args=(q,))
    p2 = Process(target=consumer, args=(q,))
    p1.start()
    p2.start()
    p1.join()
    p2.join()

```

Run it using: python queue_message.py

```

E:\24485A0503\cc_lab1\Week2>python queue_message.py
Producer sent: one
Consumer received: one
Producer sent: two
Consumer received: two
Producer sent: three
Consumer received: three
Producer sent: four
Consumer received: four
Producer sent: five
Consumer received: five
Consumer received: END
E:\24485A0503\cc_lab1\Week2>

```

Socket IPC

Socket IPC is a method of inter-process communication that enables processes to exchange data through sockets.

- Conceptually, each process is assigned a “virtual network port,” allowing them to send and receive messages.
- Socket IPC works both for processes on the same machine and for processes on different machines across a network.

Create Socket IPC:

- Create two Python files: server_socket.py and client_socket.py.

Write the following code into server_socket.py:

```

import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("localhost", 9000))
server.listen(1)
print("Server waiting...")

```

```

conn, addr = server.accept()
print("Connected:", addr)
data = conn.recv(1024).decode()
print("Server received the data:", data)
conn.send(b"Hello Client this is Server")
conn.close()//server_socket.py

```

Write the following code into client_socket.py:

```

import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("localhost", 9000))
client.send(b"Hello Server this is Client")
print("Client received:", client.recv(1024).decode())
client.close()//client_socket.py

```

Run :

In the first terminal, run the server: python server_socket.py

```

E:\24485A0503\cc_lab1\Week2>python server_socket.py
Server waiting...

```

In the

second terminal, run the client: python client_socket.py

```

E:\24485A0503\cc_lab1\Week2>python client_socket.py
Client received: Hello Client this is Server

E:\24485A0503\cc_lab1\Week2>

```

Output: the server print the client message and client print the server response.

```

E:\24485A0503\cc_lab1\Week2>python server_socket.py
Server waiting...
Connected: ('127.0.0.1', 59372)
Server received the data: Hello Server this is Client

E:\24485A0503\cc_lab1\Week2>

```

3)Publish/Subscribe (Pub/Sub) Pattern

The **publish/subscribe** pattern is a **messaging model** where **senders (publishers)** do not send messages directly to receivers. Instead, messages are sent to a **topic** or **channel**, and **receivers (subscribers)** get messages from the topics they are interested in.

How It Works

1. **Publisher sends a message:**The publisher sends data to a topic, **without knowing the subscribers**.

2. **Broker routes the message:** A message broker or messaging system handles message distribution.
3. **Subscribers receive messages:** Only subscribers who are interested in that topic get the messages.

Key Features

- **Decoupling of producers and consumers:** Publishers and subscribers operate independently.
- **Many-to-many communication:** Multiple publishers and multiple subscribers can communicate via the same topic.
- **Event-driven architecture support:** Supports systems that react to events in real-time.

Examples

- **MQTT** – Common in IoT communication.
- **Redis Pub/Sub** – Lightweight message passing for applications.
- **Apache Kafka topics** – High-throughput distributed messaging.
- **ROS topics** – Robotics message passing.
- **Google Cloud Pub/Sub** – Cloud-based messaging service.

Use Cases

- **Real-time notifications:** Chat apps, alerts, or updates.
- **IoT sensor networks:** Devices send sensor data to topics for processing.
- **Logging and monitoring systems:** Centralized log collection from multiple sources.
- **Distributed systems and microservices:** Enables services to communicate asynchronously.

Create Publish/Subscribe with ZeroMQ

Step 1: First, install the ZeroMQ Python library:

pip install pyzmq

```
E:\24485A0503\cc_lab1\Week2>pip install pyzmq
Collecting pyzmq
  Downloading pyzmq-27.1.0-cp310-cp310-win_amd64.whl.metadata (6.0 kB)
  Downloading pyzmq-27.1.0-cp310-cp310-win_amd64.whl (632 kB)
    632.2/632.2 kB 1.7 MB/s 0:00:00
Installing collected packages: pyzmq
Successfully installed pyzmq-27.1.0

E:\24485A0503\cc_lab1\Week2>
```

Step 2: Create two files publisher.py and subscriber.py.

1. Write the following into pub.py file

```
import zmq
import time
context = zmq.Context()
socket = context.socket(zmq.PUB)
socket.bind("tcp://*:5556") # Bind to port 5556
while True:
```

```
msg = f"update: time={time.time()}"
print("Publisher sent the Message:", msg)
socket.send_string(msg)
time.sleep(3)//pub.py
```

2. Write the following code into sub.py file:

```
import zmq
context = zmq.Context()
socket = context.socket(zmq.SUB)
socket.connect("tcp://localhost:5556") # Connect to publisher
socket.setsockopt_string(zmq.SUBSCRIBE, "update") # Subscribe to messages with prefix
"update"
while True:
    msg = socket.recv_string()
    print("Subscriber received the Message:", msg)//sub.py
```

Run:

In one terminal, run: python pub.py

```
E:\24485A0503\cc_lab1\Week2>python pub.py
Publisher sent the Message: update: time=1765792491.0675747
Publisher sent the Message: update: time=1765792494.068668
Publisher sent the Message: update: time=1765792497.0736818
Publisher sent the Message: update: time=1765792500.0850682
Publisher sent the Message: update: time=1765792503.090274
Publisher sent the Message: update: time=1765792506.0940435
Publisher sent the Message: update: time=1765792509.0954604
Publisher sent the Message: update: time=1765792512.0959964
Publisher sent the Message: update: time=1765792515.110926
Publisher sent the Message: update: time=1765792518.1247208
Publisher sent the Message: update: time=1765792521.1344364
```

In another terminal, run: python sub.py

```
E:\24485A0503\cc_lab1\Week2>python sub.py
Subscriber received the Message: update: time=1765792506.0940435
Subscriber received the Message: update: time=1765792509.0954604
Subscriber received the Message: update: time=1765792512.0959964
Subscriber received the Message: update: time=1765792515.110926
Subscriber received the Message: update: time=1765792518.1247208
Subscriber received the Message: update: time=1765792521.1344364
Subscriber received the Message: update: time=1765792524.1481724
Subscriber received the Message: update: time=1765792527.151407
Subscriber received the Message: update: time=1765792530.1604183
Subscriber received the Message: update: time=1765792533.1730893
```

Output :

the publisher print the messages it sends, and the subscriber receive those messages live.